

Of course. Here is a formal description of the application's architecture, summarizing the work we've done. This document is ready to be shared.

Astoria Open: System Architecture Document

Overview

The Astoria Open application is an advanced conversational AI platform designed to provide a unified, natural language interface to disparate data sources. It combines structured data from traditional SQL databases with unstructured text from document libraries, presenting a synthesized, panoramic view of the subject matter to the user. While currently configured for maritime history, it is built as a generic platform adaptable to any data domain.

1. User Features

From an end-user's perspective, the application provides the following key features:

- **Natural Language Querying:** Users can ask questions in plain English, eliminating the need for technical knowledge of SQL or database schemas.
- **Unified Data Access:** The system retrieves information from both a structured SQL database and an unstructured document library (vector database) simultaneously.
- **Synthesized Narrative Summaries:** For simple queries, the application provides a concise, AI-generated narrative that combines and summarizes the findings from all data sources.
- **Complex Analytical Capability:** For complex or ambiguous questions, the system uses a powerful AI agent to explore the database, reason about the data, and provide a direct answer.
- **Transparent & Verifiable Results:** Users can view the raw data tables and the exact SQL query that was generated by the AI, ensuring transparency and trust in the results.

- **Query History:** The interface maintains a session history of all questions asked, allowing users to easily revisit previous results.

2. Architecture Components

The application is built on a modern, multi-LLM architecture, with each component specializing in a specific task.

- **Frontend UI:** A responsive Single-Page Application (SPA) built with **React and Vite**. It provides the full user experience and runs in the user's browser.
- **Backend Server:** A high-performance **FastAPI** application running on a **Gunicorn/Uvicorn** server. It exposes the REST API that the frontend communicates with.
- **Intelligent Router:** A logic layer within the backend that analyzes incoming user queries and routes them to the most appropriate tool—either the Direct SQL Chain or the SQL Agent.
- **Direct SQL Chain:** The primary tool for simple, factual questions. It uses **Google's Gemini API** to convert a natural language question directly into an efficient SQL query.
- **SQL Agent:** A powerful, autonomous agent also powered by **Google's Gemini API**. It is used for complex, ambiguous, or multi-step queries that require reasoning, database exploration (e.g., checking table schemas), and self-correction.
- **Vector Database:** A **PostgreSQL** database supercharged with the **pgvector** extension and hosted on **Supabase**. It stores vector embeddings of unstructured text documents for fast semantic search.
- **Embeddings Model:** A **sentence-transformers** model that converts user queries and documents into numerical vectors for storage and comparison in the Vector Database.
- **Narrative Synthesizer:** A fast **Groq**-powered Llama 3 model responsible for the final step of the RAG pipeline: taking the structured results from the SQL query and the unstructured context from the vector search and synthesizing them into a cohesive, human-readable narrative.

3. Detailed Data Flow

The application's data flow intelligently adapts based on the complexity of the user's question, following one of two primary paths.

Simple Query Flow (e.g., "List 5 schooners")

- 1 The user submits the query through the React UI.
- 2 The FastAPI backend receives the request at the `/api/query` endpoint.
- 3 The **Intelligent Router** analyzes the query and identifies it as a simple, factual question.
- 4 The query is passed to the **Direct SQL Chain**.
- 5 The chain sends the query to the **Gemini API**, which returns an optimized SQL string.
- 6 The backend executes the SQL against the **PostgreSQL Database**, retrieving structured results (e.g., a list of ships).
- 7 Simultaneously, the original user query is converted into a vector by the **Embeddings Model**.
- 8 This vector is used to search the **Vector Database** for relevant unstructured documents (e.g., historical texts about schooners).
- 9 Both the structured SQL results and the unstructured vector context are passed to the **Narrative Synthesizer**.
- 10 The **Groq LLM** generates a final narrative summary.
- 11 The backend packages the narrative, the raw SQL results, and the generated SQL into a single JSON response and sends it back to the React UI for display.

Complex Query Flow (e.g., "Analyze the most common engine types")

- 1 The user submits the query through the React UI.
- 2 The FastAPI backend receives the request.
- 3 The **Intelligent Router** analyzes the query and identifies it as complex and analytical.

- 4 The query is passed to the **SQL Agent**.
- 5 The **SQL Agent (powered by Gemini)** begins a reasoning process. It might first use a tool to list the database tables, then another tool to inspect the schema of the **vessels** table to understand its columns.
- 6 Based on this exploration, the Agent formulates and executes one or more SQL queries against the **PostgreSQL Database**.
- 7 After analyzing the results, the Agent formulates a final, text-based answer to the user's question.
- 8 The backend packages the agent's final answer into a JSON response and sends it back to the React UI for display.

4. The Platform Architecture

This application was designed not just as a tool for maritime history but as a reusable **platform** for creating a natural language interface for any domain with both structured and unstructured data. The core architecture is completely data-agnostic. Open source according to Apache 2.0 license.

To adapt this platform to a new context (e.g., financial analysis, medical records, or product inventories), only the data and the AI's "persona" need to be changed:

- **The Data:** Connect the system to a new SQL database and load new, domain-specific documents into the vector database.
- **The Prompts:** Update the system prompts for the agent and the narrative synthesizer to reflect the new domain (e.g., changing the persona from a "maritime historian" to a "financial analyst").

The underlying code—the FastAPI server, the React UI, the intelligent router, and the entire data flow logic—would remain unchanged. This makes the system a powerful and flexible platform for providing a single, panoramic, and conversational view of any subject matter.